

JavaScript Introduction

- JavaScript was designed to add interactivity to HTML pages
- JavaScript is a scripting language
- A scripting language is a lightweight programming language
- JavaScript is usually embedded directly into HTML pages
- JavaScript is an interpreted language (means that scripts execute without preliminary compilation)
- Developed by Netscape Corporation known as “Live Script”
- It was later renamed to its present name by Netscape, which had a joint venture with Sun Microsystems
- JavaScript supports server-side scripting also, separately known as Livewire
- First supported in Netscape Navigator 2.0.
- Now - Internet Explorer from Microsoft, Personal Web Client 3.0 from Lotus and Mosaic 2.0 and so on.
- JavaScript was also referred to as ECMA Script. ECMA is the short form for European Computer Manufacturer's Association.

What can a JavaScript do?

- **JavaScript gives HTML designers a programming tool** - HTML authors are normally not programmers, but JavaScript is a scripting language with a very simple syntax! Almost anyone can put small "snippets" of code into their HTML pages
- **JavaScript can put dynamic text into an HTML page** - A JavaScript statement like this: `document.write("<h1>" + name + "</h1>")` can write a variable text into an HTML page
- **JavaScript can react to events** - A JavaScript can be set to execute when something happens, like when a page has finished loading or when a user clicks on an HTML element
- **JavaScript can read and write HTML elements** - A JavaScript can read and change the content of an HTML element
- **JavaScript can be used to validate data** - A JavaScript can be used to validate form data before it is submitted to a server. This saves the server from extra processing
- **JavaScript can be used to detect the visitor's browser** - A JavaScript can be used to detect the visitor's browser, and - depending on the browser - load another page specifically designed for that browser
- **JavaScript can be used to create cookies** - A JavaScript can be used to store and retrieve information on the visitor's computer

Limitations with JavaScript

- Client-side JavaScript does not allow the reading or writing of files. This has been kept for security reason.
- JavaScript can not be used for Networking applications because there is no such support available.
- JavaScript doesn't have any multithreading or multiprocessing capabilities.

JavaScript Vs Java

- JavaScript has nothing to do with Java language
- Java is purely an object-oriented language, JavaScript is just a scripting tool
- JavaScript is not compiled and executed; the client directly interprets it
- JavaScript is a freely typed language whereas Java is a strongly typed one
- Object references must exist at compile-time in Java (static binding) whereas they are checked only at runtime in JavaScript (dynamic binding)

Advantages of JavaScript

- **Less server interaction:** You can validate user input before sending the page off to the server. This saves server traffic, which means less load on your server.
- **Immediate feedback to the visitors:** They don't have to wait for a page reload to see if they have forgotten to enter something.
- **Increased interactivity:** You can create interfaces that react when the user hovers over them with a mouse or activates them via the keyboard.
- **Richer interfaces:** You can use JavaScript to include such items as drag-and-drop components and sliders to give a Rich Interface to your site visitors.
- Control the frame navigation
- Include a plug-in or a java applet inside a page
- Form validation in the client's place itself, thereby reducing the burden on the server
- Images can swap when the user moves a mouse over them,
- Calculations can be made without having to resort to a cgi script
- Javascript timer on the client to check how much time he/she takes to fill a form.
- Apart from all these advantages, javascript is very easy to learn.

JavaScript Syntax

- A JavaScript consists of JavaScript statements that are placed within the `<script>... </script>` HTML tags in a web page.
- You can place the `<script>` tag containing your JavaScript anywhere within you web page but it is preferred way to keep it within the `<head>` tags.
- The `<script>` tag alert the browser program to begin interpreting all the text between these tags as a script. So simple syntax of your.

JavaScript Syntax

- The script tag takes two important attributes:
- **language:** This attribute specifies what scripting language you are using. Typically, its value will be *javascript*. Although recent versions of HTML (and XHTML, its successor) have phased out the use of this attribute.
- **type:** This attribute is what is now recommended to indicate the scripting language in use and its value should be set to *"text/javascript"*.

JavaScript Placement in HTML File

```
<html>
```

```
<body>
```

```
<script language="javascript"  
type="text/javascript">
```

```
    JavaScript code
```

```
</script>
```

```
</body>
```

```
</html>
```

JavaScript Placement in HTML File

There is a flexibility given to include JavaScript code anywhere in an HTML document. But there are following most preferred ways to include JavaScript in your HTML file.

- Script in `<head>...</head>` section.
- Script in `<body>...</body>` section.
- Script in an external file and then include in `<head>...</head>` section.

JavaScript in <head></head> section

```
<html>
```

```
<head>
```

```
<script type="text/javascript">
```

```
function sayHello()
```

```
{
```

```
  alert("Hello World")
```

```
}
```

```
</script>
```

```
</head>
```

```
<body><input type="button" onclick="sayHello()" value="Say Hello" > </body> </html>
```

JavaScript in <body></body> section:

```
<html> <head> </head>
```

```
<body>
```

```
<script type="text/javascript">  
  document.write("Hello World")
```

```
</script> <p>This is web page body </p>  
</body>
```

```
</html>
```

JavaScript in External File

```
<html> <head>
```

```
<script type="text/javascript" src="filename.js">
```

```
</script>
```

```
</head>
```

```
<body> ..... </body>
```

```
</html>
```

JavaScript Variables and DataTypes

- JavaScript variables are "containers" for storing information.
- Variable names must begin with a letter
- Variable names can also begin with \$ and _.
- Variable names are case sensitive.
- JavaScript variables can also hold other types of data, like text values .
- In JavaScript a text like "John Doe" is called a string.
- When you assign a text value to a variable, put double or single quotes around the value.
- When you assign a numeric value to a variable, do not put quotes around the value. If you put quotes around a numeric value, it will be treated as text.

JavaScript Data Types

String, Number, Boolean, Array, Object, Null, Undefined.

JavaScript Has Dynamic Types

- JavaScript has dynamic types. This means that the same variable can be used as different types.

```
var x          // Now x is undefined
var x = 5;      // Now x is a Number
var x = "John"; // Now x is a String
```

JavaScript Strings

- A string is a variable which stores a series of characters like "John Doe".
- A string can be any text inside quotes. You can use single or double quotes:

JavaScript Numbers

JavaScript has only one type of numbers. Numbers can be written with, or without decimals:

```
var x1=34.00;    //Written with decimals  
var x2=34;       //Written without decimals
```

Extra large or extra small numbers can be written with scientific (exponential) notation.

```
var y=123e5;     // 12300000  
var z=123e-5;    // 0.00123
```

JavaScript Booleans

Booleans can only have two values: true or false. Booleans are often used in conditional testing. You will learn more about conditional testing in a later chapter of this tutorial.

```
var x=true  
var y=false
```

JavaScript Arrays

```
var cars=new Array();  
cars[0]="Saab";  
cars[1]="Volvo";  
cars[2]="BMW";
```

or (condensed array):

```
var cars=new Array("Saab","Volvo","BMW");
```

JavaScript Objects

An object is delimited by curly braces. Inside the braces the object's properties are defined as name and value pairs (name : value). The properties are separated by commas:

```
var person={firstname:"John", lastname:"Doe", id:5566};
```

```
name=person.lastname;
```

```
name=person["lastname"];
```

Undefined and Null

- **Undefined** is the value of a variable with no value.
- Variables can be emptied by setting the value to **null**.

```
cars=null;  
person=null;
```

Declaring Variable Types

When you declare a new variable, you can declare its type using the "new" keyword:

JavaScript variables are all objects. When you declare a variable you create a new object.

```
var carname=new String;  
var x=    new Number;  
var y=    new Boolean;  
var cars=  new Array;  
var person= new Object;
```

Operators

- Addition (+)
- Subtraction (-)
- Multiplication (*)
- Division (/)
- Modulus (%)
- String concatenation operator is “+”
- Pre-increment operator and post-increment operators (++)
- Pre-decrement and post-decrement operators (--)
- NOT (!)
- AND (&&)
- OR (||)
- Identity (===)
- Non identity (!=)

JavaScript Arithmetic Operators

Operator	Description	Example	Result of x	Result of y
+	Addition	<code>x=y+2</code>	7	5
-	Subtraction	<code>x=y-2</code>	3	5
*	Multiplication	<code>x=y*2</code>	10	5
/	Division	<code>x=y/2</code>	2.5	5
%	Modulus (division remainder)	<code>x=y%2</code>	1	5
++	Increment	<code>x=++y</code>	6	6
		<code>x=y++</code>	5	6
--	Decrement	<code>x=--y</code>	4	4
		<code>x=y--</code>	5	4

JavaScript Objects

- Everything" in JavaScript is an Object: a String, a Number, an Array, a Date.
- In JavaScript, an object is data, with properties and methods.
- In JavaScript, objects are data (variables), with properties and methods.

```
var txt = "Hello";
```

Methods:

```
txt.indexOf(),txt.length()
```

Accessing Object Properties

- The syntax for accessing the property of an object is:
- *objectName.propertyName*
- This example uses the length property of the String object to find the length of a string:
- ```
var message="Hello World!";
var x=message.length;
```

# Data Type Conversions

- JavaScript is a dynamically typed language
- We can convert a number in to string and vice versa
- `parseInt` method - used to convert a string value, which is holding a number data type
- `temp = parseInt("42")`
- `anoth = parseInt("54.34")`
- `Str="" + 2500;`
- `Str="" + temp;`
- Simply adding an empty quote to a number value will convert a number into string data type.

# Decision Constructs

## If...else

```
If (condition)
{
statements, if it is true
}
else
{
statements if it is false
}
```

## *For Multiple Conditions*

```
if(condition)
{
Statements
}
else if (condition) {
statements
}
```

# Sample Program

```
<script language="JavaScript">
var type;
type=prompt("enter the user type","");
if(type=="admin")
document.write("Welcome Administrator");
else if(type=="user")
document.write("Welcome user");
else
document.write("Sorry...");
</script>
```

# Switch...case

```
<script language="JavaScript">
var type;
type=prompt("Enter the user type","");
switch(type)
{
 case "admin": document.write("Welcome Administrator");
 break;
 case "user": document.write("Welcome user"); break;
 default: document.write("Sorry...");
}
</script>
```

# JavaScript Popup Boxes

- In JavaScript we can create three kinds of popup boxes: **Alert box**, **Confirm box**, and **Prompt box**.

## Alert Box

An alert box is often used if you want to make sure information comes through to the user.

When an alert box pops up, the user will have to click "OK" to proceed.

### Syntax:

```
alert("sometext")
```

## Confirm Box

A confirm box is often used if you want the user to verify or accept something.

When a confirm box pops up, the user will have to click either "OK" or "Cancel" to proceed.

If the user clicks "OK", the box returns true. If the user clicks "Cancel", the box returns false.

### Syntax:

```
confirm("sometext")
```

# Prompt Box

A prompt box is often used if you want the user to input a value before entering a page.

When a prompt box pops up, the user will have to click either "OK" or "Cancel" to proceed after entering an input value.

If the user clicks "OK" the box returns the input value. If the user clicks "Cancel" the box returns null.

## Syntax:

```
prompt("sometext","defaultvalue")
```

# JavaScript Functions

- A function is a reusable code-block that will be executed by an event, or when the function is called.
- To keep the browser from executing a script when the page loads, you can put your script into a function.
- A function contains code that will be executed by an event or by a call to that function.
- You may call a function from anywhere within the page (or even from other pages if the function is embedded in an external .js file).

# JavaScript Function Syntax

```
function functionname()
{
 some code to be executed
}
```

# JavaScript Functions

A function is a block of code that will be executed when "someone" calls it.

```
<!DOCTYPE html>
<html>
<head>
<script>
function myFunction()
{
 alert("Hello World!");
}
</script>
</head>
<body>
<button onclick="myFunction()">Try it</button>
</body>
</html>
```

# Functions With a Return Value

- Sometimes you want your function to return a value back to where the call was made.
- This is possible by using the *return* statement.
- When using the *return* statement, the function will stop executing, and return the specified value.

## Calling a Function with Arguments

- When you call a function, you can pass along some values to it, these values are called *arguments* or *parameters*.
- These arguments can be used inside the function.

```
<button onclick="myFunction('Abhishek
,Professor')">click here</button>
```

```
<script>
function myFunction(name,job)
{
 alert("Welcome " + name + ", the " + job);
}
</script>
```

# Functions With a Return Value

- Sometimes you want your function to return a value back to where the call was made.
- This is possible by using the *return* statement.
- When using the *return* statement, the function will stop executing, and return the specified value.

## Syntax

```
function myFunction()
{
 var x=5;
 return x;
}
```

# Examples

```
<html>
<head>
<script type="text/javascript">
function product(a,b)
{
return a*b
}
</script>
</head>
<body>
<script type="text/javascript">
document.write(product(4,3))
</script>
<p>The script in the body section calls a function with two parameters
 (4 and 3).</p>
<p>The function will return the product of these two parameters.</p>
</body>
</html>
```

# JavaScript For...In Statement

The for...in statement is used to loop (iterate) through the elements of an array or through the properties of an object.

The code in the body of the for ... in loop is executed once for each element/property.

## Syntax

```
for (variable in object)
{
 code to be executed
}
```

# Example

```
<html>
 <body>
 <script type="text/javascript">
 var x var mycars = new Array()
 mycars[0] = "Saab"
 mycars[1] = "Volvo"
 mycars[2] = "BMW"
 for (x in mycars)
 {
 document.write(mycars[x] + "
")
 }
 </script>
 </body>
</html>
```

# JavaScript Event

- Events are normally used in combination with functions, and the function will not be executed before the event occurs!
- By using JavaScript, we have the ability to create dynamic web pages. Events are actions that can be detected by JavaScript.
- Every element on a web page has certain events which can trigger JavaScript functions. For example, we can use the **onClick** event of a button element to indicate that a function will run when a user clicks on the button.

Examples of events:

- A mouse click
- A web page or an image loading
- Mousing over a hot spot on the web page
- Selecting an input box in an HTML form
- Submitting an HTML form
- A keystroke

# JavaScript Event

Each and every element of the form will have some restricted events associated with it

- Blur - Button, Checkbox, File Upload, Frame, Layer, Password, Radio, Reset, Select, Submit, Text, Text area, Window
- Focus - Button, Checkbox, File Upload, Frame, Layer, Password, Radio, Reset, Select, Submit, Text, Text area, Window
- Change - FileUpload, Select, Text, Text area
- Click - Button, Checkbox, Document, Links, Radio, Reset, Submit
- Double Click - Area, Document, Links
- Key Down - Document, Image, Links, Text area
- Key Up - Document, Image, Links, Text area

- Load - Document, Image, Layer, Window
- UnLoad - Document, Window
- Error - Window, Image
- Mouse Down - Button, Document, Link
- Mouse Up - Button, Document, Link
- Mouse Over - Area, Layer, Link
- Mouse Out - Area, Layer, Link
- Submit, Reset - This is for a page
- Select - Text, Text Area
- Resize - Frame, Window

# Mouse Events

Property	Description
<u><a href="#">onclick</a></u>	The event occurs when the user clicks on an element
<u><a href="#">ondblclick</a></u>	The event occurs when the user double-clicks on an element
<u><a href="#">onmousedown</a></u>	The event occurs when a user presses a mouse button over an element
<u><a href="#">onmousemove</a></u>	The event occurs when the pointer is moving while it is over an element
<u><a href="#">onmouseover</a></u>	The event occurs when the pointer is moved onto an element
<u><a href="#">onmouseout</a></u>	The event occurs when a user moves the mouse pointer out of an element
<u><a href="#">onmouseup</a></u>	The event occurs when a user releases a mouse button over an element

# Keyboard Events

Attribute	Description
<u>onkeydown</u>	The event occurs when the user is pressing a key
<u>onkeypress</u>	The event occurs when the user presses a key
<u>onkeyup</u>	The event occurs when the user releases a key

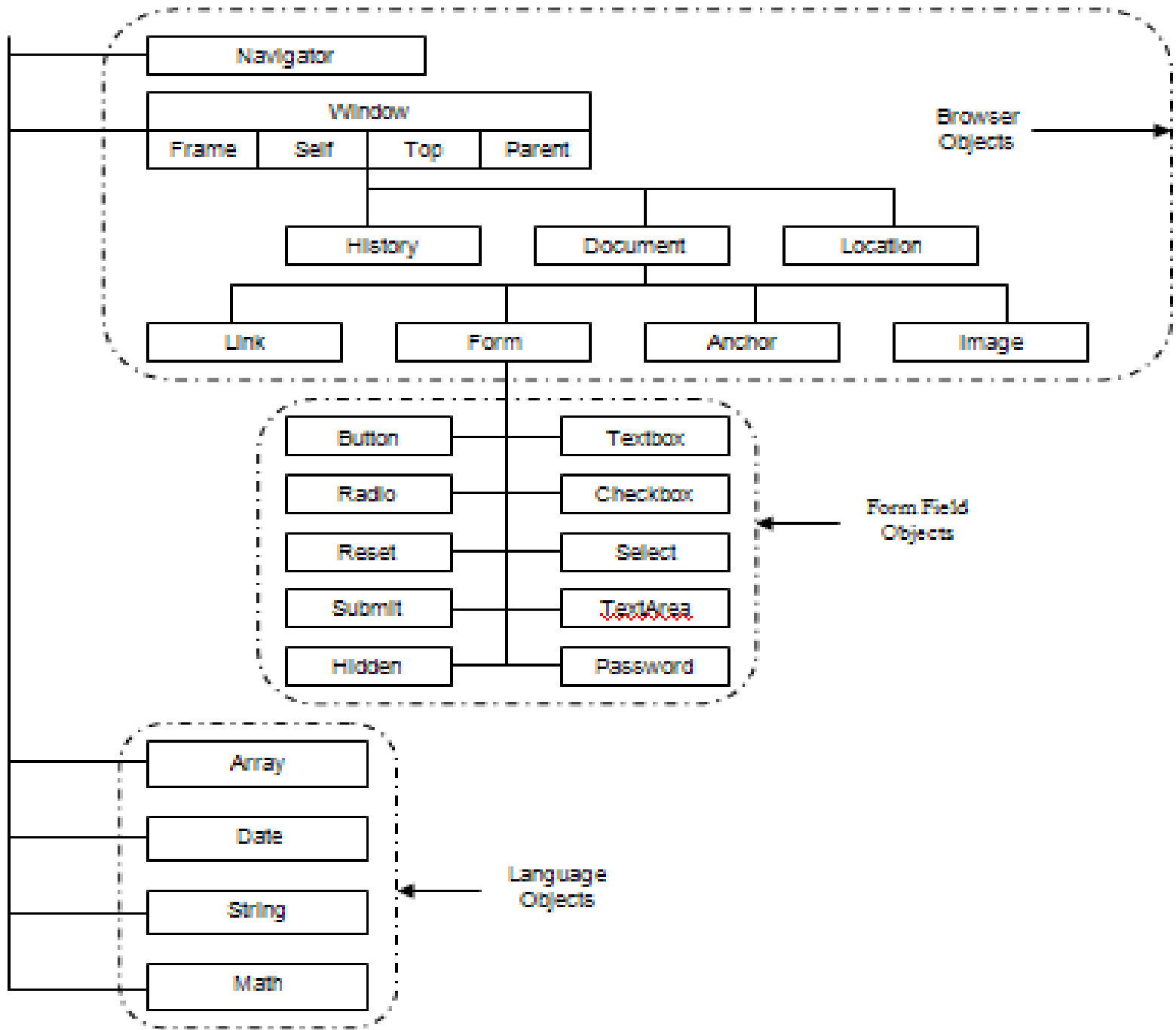
# Form Events

Attribute	Description
<u>onblur</u>	The event occurs when a form element loses focus
<u>onchange</u>	The event occurs when the content of a form element, the selection, or the checked state have changed (for <input>, <select>, and <textarea>)
<u>onfocus</u>	The event occurs when an element gets focus (for <label>, <input>, <select>, <textarea>, and <button>)
onreset	The event occurs when a form is reset
<u>onselect</u>	The event occurs when a user selects some text (for <input> and <textarea>)
onsubmit	The event occurs when a form is submitted

# JavaScript Object Model

- Models HTML documents in a very consistent and intuitive way
- Provides an interface to access, navigate, and manipulate web page
- DOM allows you to access and update the content, structure, and style of a document
- To provide more interactive content on the Web
- DOM allows HTML to be manipulated dynamically
- Each HTML tag can be accessed and manipulated via its ID and NAME attributes
- Each object has its own properties, methods, and events

- The Object Model allows you to navigate along the document tree, up and down, and to the sides
- You can use the child, parent, and sibling relationships to go from a particular point to any other point on your page tree
- Model is divided into three sub-groups
  - Language objects
  - Form-field objects
  - Browser objects



# Browser Objects

- navigator
- window
- document
- location
- history
- anchor
- link
- frame
- image

# Form-Field Objects

- Button
- Checkbox
- Radio
- Text
- Reset
- Text Area
- Password
- Select
- Submit
- Hidden

# Language Objects

- Date
- Array
- Math
- String

# Frame/Object Events

Attribute	Description
<code>onabort</code>	The event occurs when an image is stopped from loading before completely loaded (for <code>&lt;object&gt;</code> )
<code>onerror</code>	The event occurs when an image does not load properly (for <code>&lt;object&gt;</code> , <code>&lt;body&gt;</code> and <code>&lt;frameset&gt;</code> )
<code><u>onload</u></code>	The event occurs when a document, frameset, or <code>&lt;object&gt;</code> has been loaded
<code><u>onresize</u></code>	The event occurs when a document view is resized
<code>onscroll</code>	The event occurs when a document view is scrolled
<code><u>onunload</u></code>	The event occurs once a page has unloaded (for <code>&lt;body&gt;</code> and <code>&lt;frameset&gt;</code> )

# JavaScript Try...Catch Statement

- The try...catch statement allows you to test a block of code for errors. The try block contains the code to be run, and the catch block contains the code to be executed if an error occurs.

There are two ways of catching errors in a Web page:

- By using the **try...catch** statement (available in IE5+, Mozilla 1.0, and Netscape 6)
- By using the **onerror** event. This is the old standard solution to catch errors (available since Netscape 3)

# JavaScript Form Validation

- JavaScript can be used to validate data in HTML forms before sending off the content to a server.
- Form data that typically are checked by a JavaScript could be:
- has the user left required fields empty?
- has the user entered a valid e-mail address?
- has the user entered a valid date?
- has the user entered text in a numeric field?

## E-mail Validation

- The function below checks if the content has the general syntax of an email.
- This means that the input data must contain an @ sign and at least one dot (.). Also, the @ must not be the first character of the email address, and the last dot must be present after the @ sign, and minimum 2 characters before the end.

# Example

```
function validateForm()
{
 var
 x=document.forms["myForm"]["fname"].valu
e;
 if (x==null || x=="")
 {
 alert("First name must be filled out");
 return false;
 }
}
```

# Example

```
function validateForm()
{
 var
 x=document.forms["myForm"]["email"].value;
 var atpos=x.indexOf("@");
 var dotpos=x.lastIndexOf(".");
 if (atpos<1 || dotpos<atpos+2 ||
 dotpos+2>=x.length)
 {
 alert("Not a valid e-mail address");
 return false;
 }
}
```

# The Window Object

- The window object represents an open window in a browser.
- If a document contain frames (<frame> or <iframe> tags), the browser creates one window object for the HTML document, and one additional window object for each frame.

## Document Object

- Each HTML document loaded into a browser window becomes a Document object.
- The Document object provides access to all HTML elements in a page, from within a script.
- **Tip:** The Document object is also part of the Window object, and can be accessed through the window.document property.
- **Note:** The Document object can also use the properties and methods of the Node object.

# The Window Object

- The **window** object is supported by all browsers. It represent the browsers window.
  - All global JavaScript objects, functions, and variables automatically become members of the window object.
  - Global variables are properties of the window object.
  - Global functions are methods of the window object.
  - Even the document object (of the HTML DOM) is a property of the window object:
    - `window.document.getElementById("header");`
- is the same as:
- `document.getElementById("header");`

# Document Object Methods

## Document Object Methods

Method	Description
<u><a href="#">close()</a></u>	Closes the output stream previously opened with <code>document.open()</code>
<u><a href="#">getElementsByName()</a></u>	Accesses all elements with a specified name
<u><a href="#">open()</a></u>	Opens an output stream to collect the output from <code>document.write()</code> or <code>document.writeln()</code>
<u><a href="#">write()</a></u>	Writes HTML expressions or JavaScript code to a document
<u><a href="#">writeln()</a></u>	Same as <code>write()</code> , but adds a newline character after each statement

# JavaScript Date Object

## Date Object Methods

Method	Description
<u><a href="#">getDate()</a></u>	Returns the day of the month (from 1-31)
<u><a href="#">getDay()</a></u>	Returns the day of the week (from 0-6)
<u><a href="#">getFullYear()</a></u>	Returns the year (four digits)
<u><a href="#">getHours()</a></u>	Returns the hour (from 0-23)
<u><a href="#">getMilliseconds()</a></u>	Returns the milliseconds (from 0-999)
<u><a href="#">getMinutes()</a></u>	Returns the minutes (from 0-59)
<u><a href="#">getMonth()</a></u>	Returns the month (from 0-11)
<u><a href="#">getSeconds()</a></u>	Returns the seconds (from 0-59)
<u><a href="#">getTime()</a></u>	Returns the number of milliseconds since midnight Jan 1, 1970

# Math Object

## Math Object Methods

Method	Description
<u><a href="#">abs(x)</a></u>	Returns the absolute value of x
<u><a href="#">acos(x)</a></u>	Returns the arccosine of x, in radians
<u><a href="#">asin(x)</a></u>	Returns the arcsine of x, in radians
<u><a href="#">atan(x)</a></u>	Returns the arctangent of x as a numeric value between -PI/2 and PI/2 radians
<u><a href="#">atan2(y,x)</a></u>	Returns the arctangent of the quotient of its arguments
<u><a href="#">ceil(x)</a></u>	Returns x, rounded upwards to the nearest integer
<u><a href="#">cos(x)</a></u>	Returns the cosine of x (x is in radians)
<u><a href="#">exp(x)</a></u>	Returns the value of $E^x$
<u><a href="#">floor(x)</a></u>	Returns x, rounded downwards to the nearest integer
<u><a href="#">log(x)</a></u>	Returns the natural logarithm (base E) of x
<u><a href="#">max(x,y,z,...,n)</a></u>	Returns the number with the highest value
<u><a href="#">min(x,y,z,...,n)</a></u>	Returns the number with the lowest value
<u><a href="#">pow(x,y)</a></u>	Returns the value of x to the power of y
<u><a href="#">random()</a></u>	Returns a random number between 0 and 1